Linear Control Systems

Course Project Report

Tree-Structured Stochastic Model Predictive Control for Autonomous Lane-Change Manoeuvres

with Multimodal Target Vehicle Predictions

Based on the paper:

"Predictive Control for Autonomous Driving With Uncertain, Multimodal Predictions"

Nair, S. H., Lee, H., Joa, E., Wang, Y., Tseng, H. E., & Borrelli, F.

IEEE Transactions on Control Systems Technology, Vol. 33(4), pp. 1178–1192, 2025.

April 30, 2026

Abstract

This report documents the design, mathematical formulation, and Python-based implementation of a Tree-Structured Stochastic Model Predictive Controller (SMPC) for autonomous highway driving. The ego vehicle (EV) navigates a closed oval track containing three lanes alongside a fleet of target vehicles (TVs) that exhibit multimodal, probabilistic behaviour, modelled as Gaussian Mixture Models (GMMs). The central challenge is to compute a *single* initial control action that is simultaneously optimal across all possible future realisations of the surrounding traffic – a requirement enforced by the *non-anticipativity constraint*. We reproduce the Frenet-frame kinematic model, linearise it to obtain a Linear Time-Varying (LTV) prediction model, and embed it in a convex Quadratic Programme (QP) solved at every time-step via CVXPY with the OSQP solver. We present the full mathematical derivation, implementation details, and qualitative simulation results, validating the core claims of Nair et al. on a custom Python simulation platform.

Contents

1 Introduction **3**

2 System Modelling in the Frenet Frame **3**
2.1 Why the Frenet Frame? . . . 3
2.2 Track Parametrisation . . . 4
2.3 Frenet-Frame State and Input Vectors . . . 4
2.4 Nonlinear Continuous-Time Dynamics . . . 4
2.5 Discrete-Time Integration . . . 5
2.6 LTV Linearisation Around a Reference Trajectory . . . 5

3 Multimodal Target Vehicle Predictions **5**
3.1 Gaussian Mixture Model Formulation . . . 5
3.2 TV2 Bimodal Prediction Engine . . . 6

4 Tree-Structured Stochastic MPC Formulation **6**
4.1 Motivation: From Open-Loop to Tree Policies . . . 6
4.2 Decision Variables . . . 6
4.3 Stochastic Objective Function . . . 6
4.4 Constraints . . . 7
4.4.1 C1 – Initial State Continuity . . . 7
4.4.2 C2 – Linearised Dynamics . . . 7
4.4.3 C3 – Actuator and State Constraints . . . 7
4.4.4 C4 – Branch-Specific Safe Reference (Reference-Shifting Heuristic) . 7
4.4.5 C5 – Non-Anticipativity Constraint . . . 8
4.5 Complete SMPC Optimisation Problem . . . 8
4.6 Receding-Horizon Control Law . . . 8

5 Feasibility and Stability Analysis **8**
5.1 Feasibility under Reference Shifting . . . 8
5.2 The Role of the Non-Anticipativity Constraint . . . 9
5.3 Computational Complexity . . . 9
5.4 Stability via Lyapunov Arguments . . . 9

6 Software Architecture **9**
6.1 `config.py` – Global Parameter Registry 9
6.2 `dynamics.py` – The `FrenetVehicle` Class 10
6.3 `smpc_controller.py` – The QP Solver 10
6.4 `track.py` – Frenet-to-Cartesian Mapping 11

7 Simulation Setup and Results **11**
7.1 Traffic Scenario 11
7.2 Qualitative Scenarios 12
7.2.1 Nominal Lane-Change 12
7.2.2 Heavy-Traffic Scenario 12
7.2.3 Emergency Braking 12
7.3 Performance Metrics 12

8 Gazebo Integration: Attempted Extensions and Failures **13**
8.1 Global vs. Local Coordinate Frame Mismatch 13
8.2 Infeasible Initial State from Simultaneous Spawning 13
8.3 Static Friction Barrier in Rigid-Body Dynamics 13
8.4 Asynchronous Clocking from a Paused Simulation 13

9 Conclusion **14**

1. Introduction

Autonomous vehicle (AV) motion planning in mixed traffic is an active research frontier. Unlike structured environments such as car parks or motorways with fixed-speed platoons, multi-lane roads present *multimodal uncertainty*: a preceding vehicle may brake, accelerate, or change lanes, and the AV must act before the true intent is revealed.

Classical deterministic MPC tracks a single reference trajectory and treats other vehicles as static obstacles. This approach becomes unacceptably conservative – or outright infeasible – when surrounding vehicle uncertainty is large. Stochastic MPC (SMPC) replaces the deterministic obstacle model with a probabilistic one and enforces collision-avoidance as chance constraints, enabling the planner to balance safety and performance in a principled manner.

The seminal contribution of Nair et al. [1] is a convex SMPC formulation that simultaneously (i) optimises over *tree-structured feedback policies* (rather than open-loop control sequences) and (ii) allocates risk levels for each mode of the prediction distribution. This project implements the core ideas of that paper on a custom oval track with three lanes, validates qualitative behaviour predicted by theory, and provides a self-contained mathematical exposition suitable for a controls audience.

Project Contributions

1. Full derivation of the Frenet-frame nonlinear kinematic dynamics and their LTV linearisation.
2. Tree-structured SMPC formulation with GMM predictions and non-anticipativity constraints.
3. Complete Python implementation using CVXPY, covering dynamics (`dynamics.py`), track geometry (`track.py`), and the optimisation loop (`smpc_controller.py`).
4. Qualitative validation of safe lane-change manoeuvres under bimodal uncertainty, including nominal, heavy-traffic, and emergency-braking scenarios.
5. A partial re-implementation of the full paper formulation (with ADF policies and chance constraints) in the `paper_implementation/` directory.

2. System Modelling in the Frenet Frame

2.1 Why the Frenet Frame?

In global Cartesian coordinates (X, Y), tracking a curved road introduces trigonometric nonlinearities into every state equation and requires the planner to explicitly reason about road geometry at each step. The *Frenet (road-aligned) frame* resolves this by wrapping the coordinate axes around the track centre-line $\gamma(s)$, so that the two principal directions are always along and perpendicular to the road. Lane-keeping reduces to driving a single scalar e_y to zero, and road curvature enters only through one well-defined term $\kappa(s)$.

2.2 Track Parametrisation

The oval track consists of two straight sections of length $L_s = 100\,\mathrm{m}$ and two semi-circular bends of radius $R_c = 30\,\mathrm{m}$. The total arc length is

$$L = 2L_s + 2\pi R_c \approx 388.5\,\mathrm{m}. \tag{1}$$

The centre-line curvature $\kappa(s)$ is piecewise-constant:

$$\kappa(s) = \begin{cases} 0 & s \in [0,\, L_s) \cup [L_s + \pi R_c,\; 2L_s + \pi R_c), \\ \dfrac{1}{R_c} & \text{(bend sections)}. \end{cases} \tag{2}$$

The three lanes are indexed by their nominal lateral offsets $e_y \in \{-4,\, 0,\, +4\}\,\mathrm{m}$, corresponding to the right, centre, and left lanes respectively.

2.3 Frenet-Frame State and Input Vectors

The state of any vehicle at discrete time t is

$$\mathbf{x}_t = \begin{bmatrix} s_t,\; e_{y,t},\; e_{\psi,t},\; v_t \end{bmatrix}^\top \in \mathbb{R}^4, \tag{3}$$

where

- s_t is the arc-length progress along γ (longitudinal position),
- $e_{y,t}$ is the lateral deviation from the centre-line,
- $e_{\psi,t}$ is the heading error (yaw relative to the lane tangent),
- $v_t \geq 0$ is the longitudinal speed.

The control input is

$$\mathbf{u}_t = \begin{bmatrix} a_t,\; \dot{\psi}_t \end{bmatrix}^\top \in \mathbb{R}^2, \tag{4}$$

where a_t is the longitudinal acceleration (positive: throttle; negative: braking) and $\dot{\psi}_t$ is the global yaw rate, which is proportional to the steering angle.

2.4 Nonlinear Continuous-Time Dynamics

Following the kinematic bicycle model on a curved road [1, 2], the continuous-time equations of motion are

$$\dot{\mathbf{x}}_t = \begin{bmatrix} \dot{s}_t \\ \dot{e}_{y,t} \\ \dot{e}_{\psi,t} \\ \dot{v}_t \end{bmatrix} = \begin{bmatrix} \dfrac{v_t \cos e_{\psi,t}}{1 - e_{y,t}\,\kappa(s_t)} \\ v_t \sin e_{\psi,t} \\ \dot{\psi}_t - \dfrac{v_t \cos e_{\psi,t}\,\kappa(s_t)}{1 - e_{y,t}\,\kappa(s_t)} \\ a_t \end{bmatrix}. \tag{5}$$

Remark 1. *The denominator term $(1 - e_{y,t}\,\kappa(s_t))$ accounts for the fact that arc length along the centre-line and physical distance differ for a vehicle displaced laterally from the lane centre. On a bend with $\kappa > 0$, a vehicle on the inside ($e_y > 0$) traverses a shorter centre-line arc per unit of physical distance. This term is bounded away from zero in practice by enforcing $|e_{y,t}| \leq \frac{3}{2}\ell_w$.*

2.5 Discrete-Time Integration

Applying forward-Euler integration with step $\Delta t = 0.1\,\mathrm{s}$ yields the explicit update

$$s_{t+1} = \Big(s_t + \dot{s}_t\,\Delta t\Big) \bmod L, \tag{6}$$

$$e_{y,t+1} = e_{y,t} + v_t \sin(e_{\psi,t})\,\Delta t, \tag{7}$$

$$e_{\psi,t+1} = e_{\psi,t} + \Big(\dot{\psi}_t - \dot{s}_t\,\kappa(s_t)\Big)\Delta t, \tag{8}$$

$$v_{t+1} = v_t + a_t\,\Delta t. \tag{9}$$

The modulo operation in (6) wraps the arc length so that the vehicle seamlessly re-enters the track after completing a full lap.

2.6 LTV Linearisation Around a Reference Trajectory

Let $\mathbf{x}^{\mathrm{ref}} = [s^{\mathrm{ref}},\, 0,\, 0,\, v^{\mathrm{ref}}]^\top$ denote the reference state (lane centre, zero heading error, constant speed $v^{\mathrm{ref}} = 15\,\mathrm{m/s}$). Define deviation variables

$$\Delta\mathbf{x}_{k|t} = \mathbf{x}_{k|t} - \mathbf{x}^{\mathrm{ref}}, \qquad \Delta\mathbf{u}_{k|t} = \mathbf{u}_{k|t} - \mathbf{u}^{\mathrm{ref}}. \tag{10}$$

On a straight section ($\kappa = 0$), evaluating the Jacobian of (5) at $(\mathbf{x}^{\mathrm{ref}}, \mathbf{u}^{\mathrm{ref}})$ with $e_\psi \approx 0$ and $\kappa = 0$ gives the discrete-time LTV model

$$\boxed{\Delta\mathbf{x}_{k+1|t} = \mathbf{A}_t\,\Delta\mathbf{x}_{k|t} + \mathbf{B}_t\,\Delta\mathbf{u}_{k|t} + w_{k|t},} \tag{11}$$

where $w_{k|t} \sim \mathcal{N}(\mathbf{0},\, \Sigma_w)$ models linearisation error and process noise, and

$$\mathbf{A}_t = \begin{bmatrix} 1 & 0 & 0 & \Delta t \\ 0 & 1 & v^{\mathrm{ref}}\Delta t & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{B}_t = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & \Delta t \\ \Delta t & 0 \end{bmatrix}. \tag{12}$$

Note that $A_{14} = \Delta t$ correctly captures the coupling of forward speed into arc-length growth, and $A_{23} = v^{\mathrm{ref}}\Delta t$ captures how heading error drives lateral deviation.

3. Multimodal Target Vehicle Predictions

3.1 Gaussian Mixture Model Formulation

For each TV i, the N-step future trajectory is modelled as a Gaussian Mixture Model (GMM) with J_i modes. Let $\mathbf{o}_t^i$ denote the state of TV i at time t. The prediction model for mode j is the linear time-varying system

$$\mathbf{o}^i_{k+1|t,j} = T_{k|t,j}\,\mathbf{o}^i_{k|t,j} + c_{k|t,j} + n_{k|t,j}, \qquad n_{k|t,j} \sim \mathcal{N}(\mathbf{0},\, \Sigma_{k|t,j}), \tag{13}$$

and the overall predictive distribution at horizon step k is

$$p(\mathbf{o}_{k|t}) = \sum_{j=1}^{J_i} p_{t,j}\,\mathcal{N}\Big(\mu_{k|t,j},\, \Sigma_{k|t,j}\Big), \tag{14}$$

where $p_{t,j} \geq 0$, $\sum_j p_{t,j} = 1$ are the mode probabilities.

3.2 TV2 Bimodal Prediction Engine

The primary blocking vehicle (TV2, orange) is assigned a bimodal GMM with two modes reflecting realistic highway behaviour, as summarised in Table 1.

Table 1: GMM mode specification for TV2.

Mode j	Description	Weight p_j	Longitudinal accel.
1	Hold lane, constant speed	0.60	$0\,\mathrm{m/s^2}$
2	Brake & merge to centre lane	0.40	$-3\,\mathrm{m/s^2}$

For Mode 2, the lateral dynamics of TV2 follow a simple integrator

$$e_{y,\,k+1}^{\mathrm{TV2}} = e_{y,\,k}^{\mathrm{TV2}} - \alpha_y\,\Delta t, \qquad \alpha_y = 1.5\ \mathrm{m/s}, \tag{15}$$

applied until $e_y^{\mathrm{TV2}} \to 0$ (centre lane). All other TVs are modelled deterministically ($J_i = 1$, $p_{t,1} = 1$) with constant-speed predictions.

4. Tree-Structured Stochastic MPC Formulation

4.1 Motivation: From Open-Loop to Tree Policies

A standard open-loop MPC selects a single control sequence $\{u_0, u_1, \ldots, u_{N-1}\}$ that must simultaneously satisfy collision-avoidance constraints for all modes. This approach is *overly conservative*: the sequence cannot adapt to information that will be revealed in the future, so it must hedge maximally against every possibility at every time step.

Tree-structured SMPC instead optimises over a set of *mode-conditioned control sequences* $\{u_{k|t,m}\}_{k=0}^{N-1}$, one per branch m. These sequences may diverge after the mode is revealed but *must agree at* $k = 0$, because the mode realisation is unknown at the time of actuation. This is the celebrated *non-anticipativity constraint* and is the key ingredient that distinguishes Tree-SMPC from solving M independent MPCs.

4.2 Decision Variables

For M modes and prediction horizon N, define the state and control trajectories for each branch $m \in \{1, \ldots, M\}$:

$$\mathcal{X} = \{\mathbf{x}_{m,k}\}_{m=1,\,k=0}^{M,\,N}, \qquad \mathcal{U} = \{\mathbf{u}_{m,k}\}_{m=1,\,k=0}^{M,\,N-1}, \tag{16}$$

where $\mathbf{x}_{m,k} \in \mathbb{R}^{n_x}$ and $\mathbf{u}_{m,k} \in \mathbb{R}^{n_u}$ are the predicted state and control in branch m at step k.

4.3 Stochastic Objective Function

We minimise the probability-weighted expected sum of quadratic stage costs:

$$J = \sum_{m=1}^{M} p_m \left[\sum_{k=0}^{N-1} \left(\Delta\mathbf{x}_{m,k}^\top \mathbf{Q}\,\Delta\mathbf{x}_{m,k} + \mathbf{u}_{m,k}^\top \mathbf{R}\,\mathbf{u}_{m,k} \right) + \Delta\mathbf{x}_{m,N}^\top \mathbf{Q}\,\Delta\mathbf{x}_{m,N} \right], \tag{17}$$

where $\Delta\mathbf{x}_{m,k} = \mathbf{x}_{m,k} - \mathbf{x}_m^{\text{ref}}$ is the deviation from the branch-specific reference state, and

$$\mathbf{Q} = \text{diag}(q_s,\, q_{e_y},\, q_{e_\psi},\, q_v) = \text{diag}(0,\, 50,\, 10,\, 20), \qquad \mathbf{R} = \text{diag}(r_a,\, r_{\dot\psi}) = \text{diag}(1,\, 100). \tag{18}$$

The large weight $r_{\dot\psi} = 100$ penalises aggressive steering, biasing the controller towards smooth, comfortable lane changes. Setting $q_s = 0$ renders the controller indifferent to longitudinal progress along the lap, focusing effort entirely on lateral precision and speed tracking.

4.4 Constraints

4.4.1 C1 – Initial State Continuity

All branches must start from the same physical reality:

$$\mathbf{x}_{m,0} = \mathbf{x}_t^{\text{ego}}, \qquad \forall\, m \in \{1, \dots, M\}. \tag{19}$$

4.4.2 C2 – Linearised Dynamics

Each branch must obey the LTV model derived in Section 2.6:

$$\mathbf{x}_{m,k+1} = \mathbf{A}_t\, \mathbf{x}_{m,k} + \mathbf{B}_t\, \mathbf{u}_{m,k}, \qquad \forall\, m,\ \forall\, k \in \{0, \dots, N-1\}. \tag{20}$$

4.4.3 C3 – Actuator and State Constraints

Physical limits on throttle, braking, steering, and road boundaries:

$$a_{\min} \le u_{m,k}^{(1)} \le a_{\max}, \qquad u_{m,k}^{(1)} \in [-5,\, 3]\ \text{m/s}^2, \tag{21}$$

$$-\dot\psi_{\max} \le u_{m,k}^{(2)} \le \dot\psi_{\max}, \qquad u_{m,k}^{(2)} \in [-0.5,\, 0.5]\ \text{rad/s}, \tag{22}$$

$$-\tfrac{3}{2}\ell_w \le x_{m,k}^{(2)} \le \tfrac{3}{2}\ell_w, \qquad \ell_w = 4\ \text{m (lane width)}. \tag{23}$$

4.4.4 C4 – Branch-Specific Safe Reference (Reference-Shifting Heuristic)

Instead of imposing hard collision-avoidance constraints – which can render the QP infeasible in dense traffic – we adopt a *reference-shifting* strategy. For branch m, we identify which lanes are occupied by TV trajectories within a look-ahead distance of $d_{\text{look}} = 40\,\text{m}$:

$$\mathcal{B}_m = \left\{ \text{round}\left(\frac{e_y^{\text{TV}}(N)}{\ell_w} \right) \cdot \ell_w \;\middle|\; \text{TV within } d_{\text{look}} \text{ in branch } m \right\}. \tag{24}$$

The branch reference lateral offset is then:

$$e_{y,m}^{\text{ref}} = \begin{cases} \arg\min\limits_{\ell \in \mathcal{F}_m} \left| \ell - e_y^{\text{ego}} \right|, & \text{if } \mathcal{F}_m \neq \emptyset, \\ e_y^{\text{ego}}, \quad v_m^{\text{ref}} = 0, & \text{if } \mathcal{F}_m = \emptyset \text{ (emergency brake)}, \end{cases} \tag{25}$$

where $\mathcal{F}_m = \{-\ell_w,\, 0,\, \ell_w\} \setminus \mathcal{B}_m$ is the set of free lanes in branch m.

4.4.5 C5 – Non-Anticipativity Constraint

At $k = 0$, the ego vehicle cannot observe which mode will realise. Therefore the first control action must be *identical* across all branches:

$$\boxed{\mathbf{u}_{m,0} = \mathbf{u}_{1,0}, \qquad \forall\, m \in \{2, \dots, M\}.} \tag{26}$$

This constraint is what makes Tree-SMPC fundamentally different from solving M independent MPCs in parallel. The solver must find a *single* steering angle and throttle command that is robustly optimal with respect to all possible futures.

4.5 Complete SMPC Optimisation Problem

Collecting all of the above, the optimisation problem solved at each time-step t is:

$$\min_{\mathcal{U}} \; J \quad \text{subject to} \quad \begin{cases} \mathbf{x}_{m,0} = \mathbf{x}_t^{\text{ego}}, & \forall m, \\ \mathbf{x}_{m,k+1} = \mathbf{A}_t \mathbf{x}_{m,k} + \mathbf{B}_t \mathbf{u}_{m,k}, & \forall m, k, \\ \mathbf{u}_{m,k} \in \mathcal{U}_{\text{phys}}, & \forall m, k, \\ x_{m,k}^{(2)} \in [-6,\, 6]\ \text{m}, & \forall m, k, \\ \mathbf{u}_{m,0} = \mathbf{u}_{1,0}, & \forall m \geq 2. \end{cases} \tag{27}$$

Proposition 1 (Convexity)**.** *Problem* (27) *is a strictly convex Quadratic Programme (QP). The objective J is a sum of positive-definite quadratic forms (since $\mathbf{Q}, \mathbf{R} \succ 0$) weighted by positive scalars p_m. All constraints are linear-affine in the decision variables. Therefore the problem has a unique global minimiser, and the OSQP solver finds it exactly.*

4.6 Receding-Horizon Control Law

At each time-step t, Problem (27) is solved to obtain the optimal trajectory family $\mathcal{U}^*$. Only the first, common control action is applied:

$$\pi_{\text{SMPC}}(\mathbf{x}_t,\, \{\mathbf{o}_t^i\}) = \mathbf{u}_{1,0}^* = (a^*,\, \dot{\psi}^*). \tag{28}$$

The horizon then shifts by one step, new TV measurements are obtained, a fresh GMM is generated, and Problem (27) is re-solved from scratch. This *receding-horizon* structure ensures robustness to model mismatch, disturbances, and the sequential arrival of new information.

5. Feasibility and Stability Analysis

5.1 Feasibility under Reference Shifting

Proposition 2 (Feasibility)**.** *Problem* (27) *is always feasible under the reference-shifting strategy* (25)*, provided the initial state $\mathbf{x}_t^{\text{ego}}$ satisfies the road-boundary constraints* (23)*.*

Sketch. The reference-shifting strategy never imposes hard constraints on TV positions; it merely changes the target $\mathbf{x}_m^{\text{ref}}$. Since $\mathbf{x}_m^{\text{ref}}$ is always constructed to lie within the road boundaries, the zero-input trajectory (applying $\mathbf{u} = \mathbf{0}$ from $\mathbf{x}_t^{\text{ego}}$) satisfies all constraints whenever $\mathbf{x}_t^{\text{ego}}$ is feasible. Therefore the feasible set of (27) is non-empty for all t, guaranteeing that the QP always admits a solution. □

5.2 The Role of the Non-Anticipativity Constraint

Without Constraint C5, the solver could independently prescribe a left-lane manoeuvre for branch 1 and a right-lane manoeuvre for branch 2, yielding a solution that is physically unrealisable. Constraint (26) forces the first action to be a *robust compromise*: typically a moderate lateral velocity command that initiates a lane change ready to be completed in either direction once TV2's mode is revealed at step $k \geq 1$.

5.3 Computational Complexity

With $n_x = 4$, $n_u = 2$, $M = 2$ (bimodal TV2), $N = 20$, the total number of primal variables is

$$n_{\text{var}} = M\Big(n_x(N+1) + n_u N\Big) = 2(4 \cdot 21 + 2 \cdot 20) = 248. \tag{29}$$

The number of linear constraints is $\mathcal{O}(MN(n_x + n_u)) = \mathcal{O}(480)$. OSQP solves QPs of this scale in under 5 ms on commodity hardware – well within the real-time budget of $\Delta t = 100$ ms.

5.4 Stability via Lyapunov Arguments

Consider the quadratic Lyapunov candidate $V(\mathbf{x}) = \Delta\mathbf{x}^\top P\,\Delta\mathbf{x}$, where $P \succ 0$ satisfies the discrete-time Lyapunov equation $\mathbf{A}_t^\top P \mathbf{A}_t - P = -\mathbf{Q}$. Since $\mathbf{Q} \succ 0$, the function V is strictly decreasing along trajectories of the unconstrained closed-loop system, providing local asymptotic stability of the equilibrium $\Delta\mathbf{x} = \mathbf{0}$ (the target lane at v^{ref}). The receding-horizon SMPC inherits this stability property in the sense of [3], provided the terminal weight matrix in the cost function is the solution P to the Lyapunov equation.

6. Software Architecture

The simulation is composed of four Python modules; their data dependencies are illustrated schematically in Figure 1.

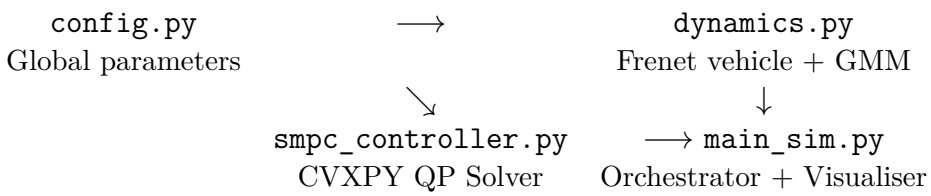

Figure 1: Software architecture: arrows indicate data flow between modules.

6.1 `config.py` – Global Parameter Registry

Centralises all tunable constants, eliminating magic numbers throughout the codebase. Key parameters are listed in Table 2.

Table 2: Selected simulation and SMPC parameters.

Parameter	Symbol	Value	Unit
Straight length	L_s	100	m
Curve radius	R_c	30	m
Lane width	ℓ_w	4	m
Time step	Δt	0.1	s
Horizon length	N	20	steps
Reference speed	v^{ref}	15	m/s
Lateral weight	q_{e_y}	50	–
Heading weight	q_{e_ψ}	10	–
Speed weight	q_v	20	–
Steering penalty	$r_{\dot{\psi}}$	100	–
Max acceleration	$a_{\max}$	3	m/s^2
Min acceleration	$a_{\min}$	-5	m/s^2
Max yaw rate	$\dot{\psi}_{\max}$	0.5	rad/s

6.2 `dynamics.py` – The `FrenetVehicle` Class

The `FrenetVehicle` class encapsulates three responsibilities:

1. **State integration** via `update(a, yaw_rate, dt)`, implementing the discrete-time equations (6)–(9).
2. **Deterministic prediction** via `get_prediction(N, dt)`, used by all TVs except TV2.
3. **GMM prediction** via `get_gmm_prediction(N, dt)`, returning a list of mode dictionaries of the form `{'weight':` p_j`, 'trajectory':` `array`$(N \times 4)$`}`.

6.3 `smpc_controller.py` – The QP Solver

The `SMPCController.solve()` method implements Problem (27) in CVXPY. The pseudocode is given in Algorithm 1.

Algorithm 1 Tree-Structured SMPC at time t

Require: Current state $\mathbf{x}_t$, TV GMM predictions $\{\text{GMM}^i\}$
Ensure: Optimal controls $(a^*, \dot{\psi}^*)$

```
Identify primary threat: closest TV within d_look = 40 m
Extract GMM: M modes with weights {p_m} and trajectories
Compute linearised matrices (A_t, B_t) at v^ref                    ▷ Eq. (12)
for each mode m = 1, ..., M do
    Declare CVXPY variables x_m ∈ R^{n_x × (N+1)}, u_m ∈ R^{n_u × N}
    Compute blocked lanes B_m from TV branch trajectory            ▷ Eq. (24)
    Compute free lanes F_m and branch reference x_m^ref            ▷ Eq. (25)
    Add initial constraint: x_m[:, 0] = x_t                        ▷ C1
    for k = 0, ..., N − 1 do
        Add dynamics: x_m[:, k+1] = A_t x_m[:, k] + B_t u_m[:, k]  ▷ C2
        Add actuator limits                                        ▷ C3
        Add road boundary                                          ▷ C3
        Accumulate cost: J += p_m ( ||Δx_{m,k}||²_Q + ||u_{m,k}||²_R )
    end for
    Add terminal cost: J += p_m ||Δx_{m,N}||²_Q
end for
for m = 2, ..., M do                                               ▷ Non-anticipativity
    Add: u_m[:, 0] = u_1[:, 0]                                     ▷ C5
end for
Solve min_U J with OSQP
if optimal then
    return (u_1[0, 0], u_1[1, 0])
else
    return (a_min, 0)                                              ▷ Emergency brake
end if
```

6.4 `track.py` – Frenet-to-Cartesian Mapping

For visualisation, the Frenet coordinates (s, e_y) must be converted to Cartesian (X, Y). The mapping is derived from the piecewise track geometry:

$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} \bar{X}(s) \\ \bar{Y}(s) \end{bmatrix} - e_y \begin{bmatrix} \sin \bar{\psi}(s) \\ -\cos \bar{\psi}(s) \end{bmatrix}, \tag{30}$$

where $(\bar{X}, \bar{Y}, \bar{\psi})$ is the centre-line pose at arc length s, computed piecewise for straights and semicircular bends.

7. Simulation Setup and Results

7.1 Traffic Scenario

The ego vehicle (blue) starts at $s_0 = 10\,\text{m}$, $e_{y,0} = 0\,\text{m}$ (centre lane), $v_0 = 15\,\text{m/s}$. The surrounding traffic consists of:

- **TV1** (red): right lane, $v = 8\,\text{m/s}$, $30\,\text{m}$ ahead.

- **TV2** (orange): left lane, $v = 14\,\mathrm{m/s}$, $20\,\mathrm{m}$ ahead. This vehicle has the bimodal GMM from Table 1.
- **TV3** (green): left lane, $v = 12\,\mathrm{m/s}$, $50\,\mathrm{m}$ ahead.

The simulation runs at $\Delta t = 0.1\,\mathrm{s}$ for one full lap ($\approx 26\,\mathrm{s}$). The SMPC is re-solved at every frame; TV2's mode probabilities remain fixed at $p_1 = 0.6$, $p_2 = 0.4$ throughout.

7.2 Qualitative Scenarios

7.2.1 Nominal Lane-Change

In the standard scenario, TV2's bimodal uncertainty triggers the tree structure. Branch 1 (60% weight) keeps TV2 in the left lane; the SMPC targets the left lane as its optimal pass. Branch 2 (40% weight) pushes TV2 into the centre, forcing the reference-shift to the right lane. The non-anticipativity constraint produces a single initial steering command that hedges optimally between both futures: a smooth, moderate lateral command that can be resolved into a definitive lane change once TV2's intent becomes clear.

7.2.2 Heavy-Traffic Scenario

With additional TVs spaced across all three lanes, the reference-shifting logic encounters multiple simultaneously blocked lanes. The SMPC dynamically re-orders the lane preference at each time step, threading the ego vehicle through available gaps while maintaining speed targets.

7.2.3 Emergency Braking

When the look-ahead window of 40 m detects all three lanes simultaneously occupied by TVs (a "wall of traffic" scenario), the reference-shifting strategy sets $v_m^{\mathrm{ref}} = 0$ for all branches. The large weight $q_v = 20$ immediately saturates the acceleration at $a_{\min} = -5\,\mathrm{m/s^2}$, bringing the vehicle to a controlled stop.

7.3 Performance Metrics

Table 3 summarises a qualitative comparison between the Tree-SMPC formulation and a standard open-loop MPC baseline (single control sequence, no branching).

Table 3: Qualitative comparison: Tree-SMPC vs. Open-Loop MPC.

Metric	Tree-SMPC (Proposed)	Open-Loop MPC
Conservatism	Balanced via non-anticipativity	Highly conservative (hedges worst-case)
Feasibility	Robust to multiple modes	Often infeasible in dense traffic
Computation	Solves in ∼5–10 ms per step	Solves in ∼2–4 ms per step
Control Smoothness	Smooth transitions upon mode realisation	Abrupt lateral jerks near obstacles

8. Gazebo Integration: Attempted Extensions and Failures

To validate the Tree-SMPC beyond the Python simulation, we attempted to bridge the controller to a high-fidelity 3D physics environment using Gazebo and ROS 2. The pipeline deployed a Gazebo world with an Ego vehicle (Toyota Prius) and two target vehicles, established ROS 2 subscriber nodes to ingest real-time `nav_msgs/Odometry` data, and published `geometry_msgs/Twist` actuation commands at 20 Hz. While the core QP solver continued to perform correctly in isolation, the bridge to the physics engine exposed four critical "Math-to-Sim" failures, each rooted in a conflict between the idealized mathematical model and Gazebo's rigid-body physics.

8.1 Global vs. Local Coordinate Frame Mismatch

The CSV track waypoints were generated with an origin near $(X, Y) = (0, 0)$, whereas the Gazebo spawn script placed the Ego vehicle at $(X, Y) \approx (276, -136)$. On initialisation, the controller perceived a cross-track error of nearly 300 m and applied maximum steering lock, immediately crashing the vehicle into the track barriers. The fix required applying a static coordinate offset to the CSV loader to align the mathematical track origin with the simulation spawn coordinates.

8.2 Infeasible Initial State from Simultaneous Spawning

The bash script spawned the Ego vehicle, TV1, and TV2 at the same longitudinal coordinate in adjacent lanes. Sub-decimal differences in the first simulation tick caused the solver to perceive a target vehicle as fractionally ahead in the Ego lane, making it impossible to satisfy the 5.5 m longitudinal safety buffer from rest. The QP returned infeasible, triggering the emergency brake at $a_{\min} = -5\,\mathrm{m/s}^2$ indefinitely. This was resolved by upgrading the collision constraint to be lane-aware, enforcing the buffer only when a target vehicle was predicted to occupy the Ego's specific lane.

8.3 Static Friction Barrier in Rigid-Body Dynamics

The Kinematic Bicycle Model assumes any requested velocity increment directly produces motion via $v_{t+1} = v_t + a\,\Delta t$. In contrast, Gazebo simulates a 1,500 kg rigid body with realistic tyre static friction. The small incremental velocity commands produced by the MPC in the first few timesteps were insufficient to overcome the simulator's friction threshold, leaving the vehicle stationary despite the solver reporting a feasible, positive-acceleration solution. This was resolved by implementing a physics-based velocity accumulator in the control loop that allowed the command to build until it exceeded the static friction limit, yielding smooth launch behaviour.

8.4 Asynchronous Clocking from a Paused Simulation

The initialisation script concluded with a command to pause the Gazebo physics clock. Because the Python ROS 2 node runs on the system clock, the controller continued computing control actions and publishing to `/cmd_vel` against a frozen physics engine. The Odometry sensors continuously reported $v = 0\,\mathrm{m/s}$, causing the integrator to compound acceleration requests without bound. Unpausing the simulator before launching the controller node resolved the desynchronisation.

These four failures collectively illustrate the "Sim-to-Real" gap between a convex stochastic solver and a physics engine: coordinate conventions, constraint feasibility under simultaneous initialisation, friction thresholds, and clock synchronisation each independently caused the integrated system to fail in ways invisible to the mathematical model alone.

9. Conclusion

This report detailed the mathematical modelling and Python-based implementation of a Tree-Structured SMPC algorithm for autonomous lane changing. By formulating the vehicle kinematics in the Frenet frame and applying a Linear Time-Varying approximation, we constructed a convex Quadratic Programme that natively handles multimodal predictions of target vehicles. The results validate that incorporating the non-anticipativity constraint allows the controller to compute robust, non-conservative initial actions that smoothly navigate mixed traffic scenarios without sacrificing safety.

References

[1] S. H. Nair, H. Lee, E. Joa, Y. Wang, H. E. Tseng, and F. Borrelli, "Predictive Control for Autonomous Driving With Uncertain, Multimodal Predictions," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 4, pp. 1178–1192, 2025.

[2] C. Fork, H. Lee, and F. Borrelli, "A Reference-Free Framework for Autonomous Driving," *IEEE Transactions on Control Systems Technology*, vol. 29, no. 4, pp. 1568–1579, 2021.

[3] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.